Implementación del Flujo de Trabajo Colaborativo con Git Flow

Líder: Bruce Derek Peña Contreras
Colaboradores:
Diego Nicolas Gutierrez Calderon
Nicolas Diaz Camargo
Cristian Steven Calderon Meneses

Ficha: 3406012
Instructora: Isaura Suarez

Resumen

En la presente práctica se implementó la metodología de trabajo colaborativo Git Flow entre dos desarrolladores para la construcción de una aplicación en Python y su integración estructurada. El proceso abarcó desde la inicialización y configuración del repositorio remoto en GitHub hasta la gestión del ciclo de vida del software utilizando ramas estándar: main, develop y feature. Se aplicó el flujo de publicación y revisión de código mediante Pull Requests, resolución de conflictos locales durante la integración y despliegue final hacia el servidor remoto.

Introducción y Objetivos

Marco Teórico

Git Flow es un modelo estandarizado de ramificación (branching model) diseñado por Vincent Driessen que facilita la gestión y el desarrollo colaborativo de software organizando el historial de commits en estructuras claras:

- master / main: Contiene exclusivamente código en estado de producción con lanzamientos (releases) estables y etiquetados (tags).develop: Funciona como la rama principal de integración

continua donde se acumulan las últimas características terminadas para la siguiente entrega.
- feature: Ramas temporales creadas a partir de develop para aislar el desarrollo de nuevas funcionalidades específicas sin alterar el código de trabajo del equipo.
- release: Ramas de preparación para empaquetar una nueva versión estable (ej. v1.0.0), permitiendo pulir detalles de documentación o metadatos antes de fusionar en master y develop.

Objetivos

Objetivo General: Aplicar el flujo de trabajo estándar Git Flow en un entorno colaborativo en pareja (líder y desarrollador), gestionando la creación de características, integración continua, control de versiones mediante lanzamientos y corrección de errores críticos en producción.

Objetivos Específicos:

- Configurar e inicializar la extensión de Git Flow en un repositorio local y remoto en GitHub.
- Desarrollar funcionalidades aisladas mediante ramas feature/ y publicarlas para revisión a través de Pull Requests.
- Ejecutar el proceso completo de empaquetamiento y publicación de versiones (Release 1.0.0) con etiquetado de versiones (--tags).

Herramientas y Entorno

- **Sistema de Control de Versiones:** Git CLI (Git Bash con extensión Git Flow integrada)
- **Plataforma de Alojamiento Remoto:** GitHub
- **Editor de Código:** Visual Studio Code
- **Lenguaje del Proyecto:** Python.
- **Equipos de Cómputo / Entorno:** 4 estaciones de trabajo locales, 1 líder y 3 colaboradores.

Procedimiento

1. Inicialización y Configuración del Repositorio (Líder de Proyecto)

1. **Creación del repositorio local y remoto:** El líder creó la carpeta del proyecto y se inicializó la estructura de Git. Se agregaron los archivos iniciales, se realizó el primer commit.

2. **Inicialización de Git Flow:** Se ejecutó la configuración base aceptando la nomenclatura por defecto (`git flow init`). La consola pasó automáticamente a la rama develop.

3. **Publicación inicial de la rama develop:** Se subió la rama develop al repositorio remoto (`git push -u origin develop`).

4. **Invitación a colaboradores:** Se enviaron y aceptaron las invitaciones de colaboración en la plataforma GitHub.

2. Trabajo Colaborativo y Desarrollo de Funcionalidades (Demás integrantes)

Sincronización por parte del colaborador: Los otros integrantes clonaron el proyecto, inicializan git flow en su entorno local y se posicionaron en la rama develop.

Creación y publicación de una rama de funcionalidad: Se inició el desarrollo de un módulo de código mediante `git flow feature`. Se implementó la lógica, se confirmó localmente y se publicó en GitHub con `git flow feature publish`.

Revisión y Fusión mediante Pull Request: Se abrió un Pull Request en GitHub hacia develop. El equipo revisó y aprobó los cambios en la interfaz web.

Cierre de la rama de funcionalidad en el entorno local: Una vez aprobado el, cada desarrollador actualizó su entorno local (git pull origin develop) y cerró la característica con `git flow feature finish`.

3. Preparación y Despliegue de Lanzamiento (Release)

Inicio de la versión (1.0.0): Se ejecutó `git flow release start 1.0.0`.

Actualización de la documentación: Se editó el archivo README.md indicando el nuevo alcance y se confirmó.

Finalización de la Release: Se finalizó con `git flow release finish 1.0.0`, integrando los cambios en master y develop y creando la etiqueta correspondiente.

Envío de la versión al servidor remoto: Se enviaron los cambios y las etiquetas (`git push origin master --tags` y `git push origin develop`).

Resultados

- **Estructura Organizativa de Ramas:** Se logró un repositorio con ramas de larga duración (`main` y `develop`) sincronizadas, manteniendo el código de producción limpio e independiente de los avances en desarrollo.
- **Trazabilidad de Cambios:** El historial de GitHub refleja las fases de desarrollo aislado en ramas `feature/`, la preparación de la versión `v1.0.0` mediante `release/`.
- **Despliegue Controlado:** La inclusión de etiquetas de versión (`tags`) permitió identificar de forma explícita el estado funcional del proyecto en cada entrega.

Discusión de Resultados

A diferencia del flujo de trabajo estándar en una única línea o mediante pushes directos a la rama principal (donde errores o *pushes* forzados pueden comprometer el historial del equipo), **Git Flow** establece una barrera de protección rígida. La obligatoriedad de pasar el desarrollo por `develop` e integrar a `main` mediante revisiones por *Pull Request* garantiza que el código en producción permanezca inalterado mientras se resuelven los conflictos de fusión de manera segura en el entorno de desarrollo.

Asimismo, la gestión de parches urgentes mediante ramas `hotfix` demostró ser eficiente, ya que permite corregir un error en producción

sin necesidad de desplegar características incompletas que aún se encuentren en fase de trabajo dentro de `develop`.

Conclusiones

1. **Estandarización y Aislamiento:** Git Flow proporciona una arquitectura sólida para proyectos colaborativos, asegurando que las nuevas funciones se desarrollen en entornos totalmente aislados de la rama de producción.
2. **Gestión de Versiones Clara:** El uso formal de ramas `release` y la creación automática de `tags` simplifica la trazabilidad del software y la preparación de entregables para el usuario final.
3. **Resolución Estructurada de Incidentes:** Se comprobó que el mecanismo de `hotfix` es la vía adecuada para solucionar fallos urgentes en producción, pues automatiza la sincronización del parche tanto en `master` como en `develop` simultáneamente.
4. **Buenas Prácticas en Equipo:** La integración de Git Flow junto con la revisión de código vía *Pull Requests* en GitHub minimiza la aparición de colisiones de código desastrosas y fomenta la validación constante entre los miembros del equipo.

Repositorio

https://github.com/Bruce-coder17/Clase-22-10-26